Rebuild the evidence, then explain it

11 October 2026 | C003a reproduction and teach-back | Synthetic examples only

Sidiq review status: Not yet reviewed. Studied October 11; integrated October 12.

Research question and hypothesis

Can we reproduce the whole-year parser evidence from source in an empty build directory, and explain why rejection is different from a zero-year result? Hypothesis: fresh executables will match the published outputs, while the deliberately wrong adapter will still report three mismatches.

This completes the reproduction slot originally dated October 10. Yesterday's failure lesson was finished locally on a zero-budget day and integrated today. Today's new learning is reproduction, not another parser extension. Consolidation follows separately; no personal teach-back is assumed.

Beginner explanation and terms

Source is the human-readable C++ program. A compiler produces an executable. Re-running an old executable checks that file; rebuilding checks whether today's source can produce working behaviour. A fixture is a fixed input or expected output. Reproduction repeats a documented method and compares observations with the recorded expectations. It does not make those expectations automatically right.

Our parser accepts nonempty ASCII decimal digits representing zero through the target int maximum. Leading zeros are allowed. Spelling is checked before range. Invalid text returns no years; it must not silently become zero. Zero itself is a valid horizon and returns the original principal.

Mathematics and hand prediction

With principal P = 1000 money units and effective annual rate r = 0.05, B(n) = P * (1+r)^n for nonnegative whole model years n. Work it out before looking at the executable: first interest is 1000 * 0.05 = 50, so B(1) = 1050. Next interest is 1050 * 0.05 = 52.50, so B(2) = 1102.50. At n = 0, no interest period occurs and B(0) = 1000. Rejection is not B(0); it means no calculation.

Complete text	Prediction	Reason
002	2 years; 1102.50	Leading zeros preserve value
0	0 years; 1000.00	Valid lower boundary
2.5	Reject; no balance	Dot violates digit-only spelling
2.0	Reject; no balance	Numerical wholeness does not change grammar

Trace the existing C++ flow

1. parse_whole_years(text) receives the original string. The const reference lets the function inspect that string without making a copy or changing it. No floating conversion happens first.

2. The empty check and character loop enforce spelling. Each character must be between '0' and '9'. The dot in 2.5 fails here, before an integer or financial balance can be returned.

3. std::from_chars performs checked decimal conversion. Its error code and end position are checked; a digit-only value beyond the int maximum raises a range error. The spelling rule has already passed.

4. Only a returned integer enters compound_balance. An invalid_argument catch in the demonstration prints rejection and returns from that case. It does not supply a substitute zero to the calculation.

Reproduction protocol and implementation

All inputs are existing synthetic fixtures; there is no historical period, fitting or market data. The reproduce_years_evidence.py recipe creates an empty temporary directory, compiles six executables with C++17 and warnings as errors, runs each evidence path twice, and removes the temporary binaries. It uses unchanged published C++ sources. No new language or strategy feature is introduced.

The six executables are savings tests, parser tests, the parser/savings demonstration, the byte probe, and correct/wrong fractional examples. The probe is passed to the existing Python boundary oracle. Checks compare exact stdout bytes, stderr bytes and exit status; compilation errors or timeouts fail the recipe. The wrong example must exit 1 with three mismatches. Arbitrary nonzero failure is not enough.

The recipe also recomputes interest with Python Decimal and records source hashes, compiler identity and the published baseline commit. It is a local verification aid, not a new production interface. This is a fresh build in the same installed environment, not an independently provisioned machine.

Observed checks and results

Check	Observed result
Fresh temporary build	Six executables; warning-clean Apple Clang 21 C++17
Existing C++ tests	66 savings checks and 30 parser checks pass twice
Boundary oracle	75 cases per run; both JSON outputs match archive
Savings demonstration	Both runs match the published output bytes
Fractional example	Both runs: two acceptances, three rejections
Wrong-adapter control	Both runs: three mismatches and exact exit 1
Independent hand ledger	Decimal credits 50.00 and 52.50; balance 1102.50

Today's integration also passed full make check: 608 Python tests, Ruff lint and 51 formatted files, strict mypy on 21 source files, the C++ checks, 75 oracle comparisons and four output fixtures. Dependency consistency passes. Those checks are shared integration evidence, not extra new tests. The fresh temporary reproduction passed on its first run. Test counts are not coverage percentages.

Reproduce with .venv/bin/python scripts/reproduce_years_evidence.py from the repository root. Original evidence: experiments/years_parser/reproduction-2026-10-11.json. Current checks run fresh; reproduction-integration.json records provenance. Run without -O so assertion checks remain active.

Costs, assumptions and failure cases

The savings model assumes fixed annual rate, reinvestment and no interim cash flows. Fees, taxes and inflation are omitted and can reduce real wealth. No spread, turnover, market return or financial loss is estimated here. Fractional-year accrual needs its own convention; casting is not that convention.

A stale binary, changed fixture or broken expected-error check can undermine reproduction. This recipe avoids existing build outputs, compares archived bytes and checks the intended failure status. Still, a shared faulty contract can make multiple methods agree. Finite tests cannot prove all-input safety. No length cap, external CLI hardening, cross-platform guarantee or personal mastery is established.

Primary sources consulted on 11 October

Python's official subprocess documentation explains captured output, process return codes and timeout behaviour. The recipe compares all three observable outcomes instead of accepting any failed process. https://docs.python.org/3/library/subprocess.html#subprocess.run

Microsoft's conversion reference explains that floating-to-integer conversion discards a fractional part toward zero, subject to representability. This explains the existing negative control; it does not define the project's accepted spelling policy. https://learn.microsoft.com/en-us/cpp/cpp/standard-conversions?view=msvc-170

Microsoft's string-functions reference describes stod and the first unconverted position. Consuming fractional text fully still does not meet our whole-year grammar. https://learn.microsoft.com/en-us/cpp/standard-library/string-functions?view=msvc-170#stod

Interview explanation

I rebuilt the existing parser evidence in an empty temporary directory and compared exact outputs, diagnostics and exit status against the recorded fixtures. The deliberate wrong adapter still failed in the expected way. This supports reproducibility on the tested environment; it is not proof that the contract is right for every application or evidence of investment performance.

Progress, publication and next task

October 11's draw of 1 was used by 0a51f14, integrating the prior failure lesson. Reproduction was saved locally that day. On October 12, its 17 input and five artifact hashes matched; the recipe was integrated and rebuilt all six executables. Full checks passed again with 52 formatted files. Original study results stay dated October 11; publication uses the actual October 12 commit date.

Next eligible learning is consolidation of C003a evidence and a readiness decision, with weekly synthesis handled separately. Do not repeat reproduction because its tracked status awaits integration. C003a/C003 remain in progress; C003b, vectors/statistics and the ledger retain prerequisites. Preserve Q004a lawful-access/sample and Q005b integration gates before empirical use. No input blocks synthetic continuation. Review stays Not yet reviewed until Sidiq says otherwise. Stop new research after December 31 and request the next programme scope; the daily schedule stays 08:00 Europe/London.

Check your understanding

1. Why does rebuilding add evidence beyond running an existing executable again?

2. Why is rejecting 2.5 different from calculating a zero-year balance?

3. Why must the wrong-adapter check inspect output as well as a nonzero exit status?

4. What does matching an archived fixture still fail to prove?

Suggested answers

1. It checks that the available source and compiler can produce the tested behaviour. An old binary could have come from different source. A fresh local build still shares the installed environment.

2. Rejection yields no horizon and no balance. Zero is a valid horizon with balance 1000 here. Replacing an error with zero silently answers a different question.

3. A crash or unrelated failure could also return nonzero. Three exact mismatch diagnostics and exit 1 establish that this deliberate fault was detected for the intended reason.

4. It does not prove the fixture or contract is correct, cover every input, demonstrate another platform works, establish realistic investment results, or confirm Sidiq's personal understanding.